

C# BACKEND TRACK | OOP PHASE

Task 1

System Analysis Task

Think about the business. Extract the models. Assign the sources.

Codeline Bootcamp · Backend Phase · OOP Session 3 Task

SOURCE TAG LEGEND

Every property of every class model gets its value from somewhere. Before writing any code, you must decide **where** each value comes from. Use one of these five source tags to label each property in your analysis:

System Generated	System Generated	The system assigns this value automatically. The user never types it. Example: an auto-incremented ID calculated as List.Count + 1.
User Input	User Input	The user types this value directly at the keyboard when prompted by the system.
Default Value	Default Value	A fixed value set by the code when the object is created. The user never chooses it. Example: isBooked = false, status = "Active".
Calculated	Calculated	Derived automatically from another object already in memory — no user prompt. Example: appointment date copied from a chosen slot, fee copied from a doctor.
From List	From List	The user selects a value from a list of options displayed on screen. Example: choosing a doctor ID from a printed list of doctors.

Domain 1 — Library Management System

Extract everything yourself

Read the following description of a real library carefully. Your job is to go through the 6-phase process and fill in all the blank tables below. Nothing has been done for you — you identify the entities, you decide the properties, and you assign the source tag for each one.

A public library allows community members to register and borrow books. Each book has a title, author, genre, and a fixed number of physical copies in stock. A registered member can borrow an available book copy and must return it within 14 days. If the book is returned late, the system calculates a fine based on how many days overdue it is. A member can also reserve a book that is currently fully borrowed so they are notified when it becomes available. The librarian needs to see all currently active loans, view all overdue loans sorted by how late they are, and get a report of the most-borrowed books ranked by total borrow count.

Step 1 — Identify the Entities (Phases 1 & 2)

From the description above, identify the main real-world objects (classes) the system must track. Write each entity name, its properties, the data type for each, and the source tag that describes how each property gets its value.

Entity Name: Book		
Property Name	Data Type	Source Tag
BookId	string	System Generated
Title	string	User Input
Author	string	User Input
Genre	string	User Input
TotalCopies	int	User Input
AvailableCopies	int	Calculated
BorrowedCopies	int	Calculated

Notes:

- $\text{AvailableCopies} = \text{TotalCopies} - \text{borrowed copies}$
- $\text{BorrowedCopies} = \text{TotalCopies} - \text{AvailableCopies}$

Entity Name: Member		
Property Name	Data Type	Source Tag
MemberId	string	System Generated
Name	string	User Input
Email	string	User Input
Email	string	User Input
PhoneNumber	string	User Input
RegistrationDate (membership date)	DateTime	Default Value (current date)

Entity Name: Borrowing		
Property Name	Data Type	Source Tag
BorrowingID	string	System Generated
MemberId	string	From List
BookId	string	From List
BorrowDate	DateTime	System Generated
DueDate	DateTime	Calculated
ReturnDate	DateTime	System Generated
IsReturned	bool (false)	
OverdueDays	int	Calculated
FineAmount	double	Calculated

Notes:

- DueDate = BorrowDate + 14 days
- OverdueDays = days after due date
- FineAmount based on overdue days

Entity Name: Reservation		
A member asks to hold a book for them when it is currently not available.		
Property Name	Data Type	Source Tag
ReservationId	string	System Generated
MemberId	string	From List
BookId	string	From List
ReservationDate	DateTime	System Generated
IsActive(Is still waiting)	bool	Default Value (true)

If you identify more than 4 entities, continue on the back of this sheet or add extra rows above.

Step 2 — Describe the Entity Relationships (Phase 1 continued)

How do the entities connect to each other? Describe the full cycle in plain English — who does what, which entity gets created, and what changes in other entities as a result.

1. Member Registration

During member registration, the user provides personal details such as name, email, and phone number. The system generates a unique MemberId automatically and assigns the current date as the registration date. A new Member entity is then created and stored in the system, allowing the member to interact with library services such as borrowing and reserving books.

2. Adding Books

When adding a book, the librarian inputs details such as title, author, genre, and total number of copies. The system generates a unique BookId automatically. The AvailableCopies property is calculated based on the total number of copies and initially equals TotalCopies. The BorrowCount is also initialized to zero. The created Book entity is then stored in the system and becomes available for borrowing.

3. Borrowing a Book

- A Member wants to borrow a book
- The system checks:
 - If the Book has available copies

If there are available copies:

- A new borrowing entity is created with:
 - **MemberId** → From Member
 - **BookId** → From Book
 - **BorrowDate** → current date
 - **DueDate** → BorrowDate + 14 days
 - **IsReturned** → false
- It connects:
 - One Member
 - One Book
- The system sets:
 - BorrowDate = today
 - DueDate = BorrowDate + 14 days
- The Book is updated:
 - AvailableCopies decreases
 - BorrowCount increases
- Result:
 - The Loan is stored in the system
 - The Member now has an active loan
 - The Book has fewer available copies
- **If there are no available copies will guide the user to make a reservation**

4. Returning a Book

- Member returns the book
- The system finds the active borrowingId(isReturned = false), bookId and memberId
- Update the Loan entity:
- The system updates the Loan:
 - **ReturnDate** → set to current date

- **IsReturned** → set to true
- Check if the book is returned late (ReturnDate > DueDate ?)
 - If NOT late:
 - No fine
 - Process ends
 - If late:
 - Calculate overdue days: ReturnDate – DueDate
 - Calculate fine= OverdueDays × fine per day

Update the Book entity: AvailableCopies = AvailableCopies + 1

- Check Reservations:

If there are active reservations:

 - Find the first reservation (oldest) with the matched bookId
 - Notify that member
 - Mark reservation: IsActive = false

5. Reserving a book:

A Reservation is created when a member wants a book, but all copies are currently borrowed.
AvailableCopies == 0

- Member requests a book:
 - The system creates a new Reservation with:
 - **ReservationId** → system generated
 - **MemberId** → who is waiting
 - **BookId** → which book
 - **ReservationDate** → current date
 - **IsActive** → true (still waiting)

Note: should serve users in FIFO manner to solve the issue If multiple members reserve the same book.

6. Tracking library activities

Tracking borrowing activity means the system keeps records of how often books are borrowed to determine the Most popular books. Also, tracking what is currently happening with loans.

- tracking how many times the book has been borrowed:
 - Knowing borrowed copies of each book can sort them in descending order to determine the best 5 books.
- Active Loans Tracking:
 - Loans where IsReturned = false determine how many books are still with members.
- Overdue Loans Tracking
 - The system checks: DueDate and ReturnDate
 - If true there is a delay means the loan is overdue and need to notify the member.

Step 3 — List the Services (Phase 4)

From the description, identify every service (function) the system needs. Write at least 8 services — give each a name and a one-line description of what it does.

#	Service Name	What it does
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

Domain 2 — Hotel Management System

Assign the source tags

The class models and all their properties are already defined for you below. Your job is to **fill in the Source Tag column** for every single property. Use the five source tags from the legend on page 1: System Generated, User Input, Default Value, Calculated, From List.

A hotel allows guests to register and book rooms for a stay. The hotel has different room types (Single, Double, Suite) each with its own nightly price. Each room can only have one active booking at a time. When a guest checks in, a check-in record is created. The total price of a stay is calculated from the number of nights and the room's nightly rate. When a guest checks out, the room becomes available again. A booking can also be cancelled before check-in, which frees the room immediately.

Your task: For each property in the tables below, write the correct source tag in the empty column on the right. Think carefully — ask yourself: does the user type this? does the system generate it? is it copied from another object? does it have a fixed starting value? does the user pick it from a list?

Guest		
Property Name	Data Type	Source Tag (fill in)
<code>guestId</code>	<code>int</code>	System Generated _____
<code>guestName</code>	<code>string</code>	User Input _____
<code>guestPhone</code>	<code>string</code>	User Input _____
<code>guestEmail</code>	<code>string</code>	User Input _____
<code>guestNationality</code>	<code>string</code>	User Input _____
<code>registrationDate</code>	<code>string</code>	Default Value (current date)

Room		
Property Name	Data Type	Source Tag (fill in)
<code>roomId</code>	<code>int</code>	System Generated
<code>roomNumber</code>	<code>string</code>	User Input
<code>roomType</code>	<code>string</code>	User Input
<code>floorNumber</code>	<code>int</code>	User Input
<code>nightlyPrice</code>	<code>decimal</code>	From List (<i>based on room type</i>)
<code>isAvailable</code>	<code>bool</code>	Default Value (<i>true initially</i>)

Booking		
Property Name	Data Type	Source Tag (fill in)
<code>bookingId</code>	<code>int</code>	System Generated
<code>guestId</code>	<code>int</code>	From List
<code>roomId</code>	<code>int</code>	From List
<code>checkInDate</code>	<code>string</code>	User Input
<code>checkOutDate</code>	<code>string</code>	User Input
<code>numberOfNights</code>	<code>int</code>	Calculated checkOutDate - checkInDate
<code>totalPrice</code>	<code>decimal</code>	Calculated
<code>status</code>	<code>string</code>	Default Value (<i>Active</i>)

CheckInRecord		
Property Name	Data Type	Source Tag (fill in)
<code>recordId</code>	int	System Generated _____
<code>bookingId</code>	int	From List _____
<code>guestId</code>	int	From List _____
<code>roomId</code>	int	From List _____
<code>actualCheckIn</code>	string	System Generated (current _____)
<code>actualCheckOut</code>	string	System Generated (when checkout happens) _____
<code>finalAmount</code>	decimal	Calculated _____

Step 2 — Describe the Entity Relationships

Now that you have studied the four class models, describe in your own words how they connect to each other. Who creates what? What changes when a booking is made? What happens when a guest checks out?

1. Guest Registration:

- A person enters their details:
 - Name, Email, Phone, Nationality
- The system:
 - Generates a `guestId`
 - Sets `registrationDate` = today
- Result:
 - A new Guest entity is created
- Why this matters:
 - The guest cannot book a room without being registered

3. Room Setup:

Note: Rooms are already created by the system (or admin).

Each Room has:

- Room number
- `roomType` (Single / Double / Suite)
- `nightlyPrice` (based on type)
- `isAvailable` = true (initially)

4. Making a Booking

- Guest selects a room
 - The system checks:
 - `isAvailable` == true ?

If room is available:

A new Booking entity is created with:

`guestId` → from Guest

`roomId` → from Room

`checkInDate` → user input

`checkOutDate` → user input

System calculates values:

`numberOfNights` = `checkOutDate` - `checkInDate`

`totalPrice` = `numberOfNights` × `nightlyPrice`

Room state changes
 isAvailable = false

5. Before Check-in (Optional Cancellation)

If guest cancels booking:
 status = Cancelled
 isAvailable = true
 Room becomes free again immediately

6. Check-in Process (Start of Stay)

What triggers this:
 Guest arrives at hotel

Create CheckInRecord:

- A new CheckInRecord is created:
 - bookingId → links to booking
 - guestId → identifies guest
 - roomId → identifies room
 - actualCheckIn → current date/time

7. Check-out Process (End of Stay):

- Update CheckInRecord: actualCheckOut = current date
- Calculate final amount: finalAmount = numberOfNights × nightlyPrice

- Update Room
 isAvailable = true
 - This is critical:
 Room becomes bookable again.

- Update Booking: status = Completed

Step 3 — List the Services

Based on the description and the class models above, list all the services (functions) this hotel system needs. Give each a name and a one-line description.

#	Service Name	What it does
1		
2		
3		
4		
5		
6		
7		

#	Service Name	What it does
8		
9		